Operational Security Review

Performed by <Indigo>
For: <REDACTED>
Date: 2/18/2022

Version 1.4

Review Summary

This is a report of the review of the operational security of <REDACTED> Domains. The review content has been split into multiple security domains, with a general evaluation of the security posture of the company and tables describing discrete security risks that have been investigated for each domain.

Each section contains an assessment of security standing for that specific domain, with general recommendations and a list of potential issues that were reviewed. All content of this report is based on asset catalogs, onboarding documentation, public resources, and employee comments provided by <REDACTED>. This review is as precise and accurate as that information.

We have focused on the most sensitive and impactful services and operations in this review, but an external audit should always be supplemented by an exhaustive internal review of all company practices and service configurations, and by continued work on all teams at <REDACTED> to follow good operational security principles.

Below are some highlights of our findings. Please read on to the individual operational security domain sections for complete details.

Report Highlights:

- **High-level recommendations:**
  - Reduce Google Cloud and GitHub access.
  - Enforce the principle of least privilege by reducing employee permissions on a need-to-know basis for all services.
  - Sign and encrypt your emails with S/MIME.
  - Establish good baseline company security training that covers phishing, workstation security, and secure data handling.
  - Put in place multi-party approval systems for critical business and technical operations.
- **Optional areas of improvement**:
  - Commission an external penetration test of all external-facing assets.
  - Perform regular internal assessments of operation security posture to ensure you continue to adequately mitigate operational security risks.
  - Perform availability, security incident response, and disaster recovery testing and game days.
  - Hire security personnel. Audits are good tools for pointing out point-in-time issues, but these are bound by time, access permissions, and scope. Security should be a continuous, ongoing effort at <REDACTED>.
    - Application security is especially important.

Table of Contents

Domain Security Posture Overview

Domain	Areas of Improvement	Overall Risk Level
Application Development and Deployment	Deployment permissions, Git commit permissions, Developer Non-repudiation	*CRITICAL*
Service Configurations	Google Cloud overly broad permissions	*CRITICAL*
Communications	Encryption & signing of emails, Secure external file handling	*MEDIUM*
Secrets and Access Management	Enforce the principle of least privilege, Password manager improvements, Non-prod secret handling	*MEDIUM*
Insider Threats	Multi-party approvals, Detective controls, Immediate (complete) access revocation	*MEDIUM*
Social Engineering, Phishing Prevention, Information Leakage	Phishing awareness training	*MEDIUM*
Endpoint Security	Basic security training	*LOW*

Domain Table Column Descriptions

- **Status** - The mitigation status of the security issue
  - MITIGATED
    - The issue was found to be sufficiently mitigated at <REDACTED>
  - UNMITIGATED
    - The issue is not mitigated, but risk is minor; mitigation is *recommended*
  - PARTIALLY MITIGATED
    - The issue is only partially mitigated; and requires additional security controls be implemented to fully mitigate the risk
  - AT RISK
    - The issue is not mitigated, and risk is significant; mitigation is necessary
  - IN PROGRESS
    - Mitigation of the issue is confirmed to be in progress
  - RESOLVED
    - The issue has been reported and mitigated during the creation of this report
- **Issue** - A description of the potential security issue
- **Control** - The recommended security control to mitigate this issue
- **Comments** - Supplemental information on how to implement controls, why the issue is a security risk, details of existing mitigations, and specifics on how <REDACTED> is affected by the issue
- **Impact** - The perceived level of damage that exploit of this issue would cause to <REDACTED>

Communications

Communications security is the number one point of failure for companies. Phishing attacks, employee impersonation, insider threats, and insecure communications transport methods all add significant security risk.

Google workspaces is used to provide Gmail access to employees. Gmail does have a good baseline security, but does not provide guarantees of message authenticity or end-to-end encryption. Authenticity verification of emails is very important to prevent phishing attacks.

Status	Issue	Control	Comments	Impact
AT RISK	Intake from external sources (bug reports, partner messages, file sharing, etc.)	Training on untrusted communication handling, provide a sandbox for viewing external files[1]	This is the #1 method for targeted malware delivery and presents a high risk due to the ease of exploit	*HIGH*
MITIGATED	Collaboration and secure file transfer/distribution	Utilize a secure file sharing service with strong access control	Google Drive and Docsend are used to securely transmit files	*HIGH*
AT RISK	Integrity and authenticity of internal emails is not guaranteed	All internal emails should be signed with a certificate from a trusted CA	Google Workspaces offers hosted S/MIME[2], or employees can set them up in clients individually	*MEDIUM*
AT RISK	Integrity and authenticity of external emails is not guaranteed	External emails signed with a certificate from a trusted CA, or otherwise verify email content[3]	Outgoing emails should be S/MIME signed, when possible. Content of unsigned incoming emails should be verified over a secure channel	*MEDIUM*
UNMITIGATED	Confidentiality of internal emails is not guaranteed	All internal emails should be end-to-end encrypted	Google encrypts emails in transit, but does not perform end-to-end encryption. S/MIME does provide E2E	*LOW*

Status	Issue	Control	Comments	Impact
AT RISK	Confidentiality of external emails is not guaranteed	All external emails should encrypted, or communicate only non-sensitive information	Sensitive information should not be transmitted via email. Authenticated file share should be used	*LOW*
UNMITIGATED	Email signing certificate revocation	Enable immediate email certificate revocation/distrust	Hosted S/MIME effectively mitigates this. Otherwise, a CA with easy revocation support should be used.	*LOW*
MITIGATED	Unencrypted chat for sensitive communications	Use and end-to-end encrypted chat software	Slack sufficiently encrypts chat data	*LOW*
UNMITIGATED	Lax chat access control rules	Restrict access to chat channels on a need-to-know basis, default chat groups to private by default, segregate slack channels by team	Enforcing the principle of least privilege on slack channels helps prevent unnecessary leakage of sensitive information	*LOW*

[1] A disconnected VM with an image that is refreshed on boot is an adequate, simple sandbox (download attachments to the VM and then disconnect network access).
[2] https://support.google.com/a/answer/6374496 - Certificates can be provisioned from any major certificate provider.
[3] Where signature verification is not feasible, authoritative or business critical information should be transmitted or verified via an alternate channel to email. E.g. verifying email contents over a pre-established secure chat channel, or phone call.

Endpoint Security

Company assets are only as secure as the endpoint machines used to access them. An “endpoint” refers to any device connected to a user (employee), that is used to access company resources (internal email, shared files, wikis, etc.).

Endpoint security is largely in the hands of users, which supports the need for good standardized security training. Basic training, in the form of text or video-based periodic training, and/or setup requirements and guidelines are highly important. <REDACTED> does not have significant security training at onboarding, or on a periodic basis. Outside of training, it is important to manage employee devices to enforce secure configurations and provide baseline security controls. The Rippling platform is used to provide this.

Employee devices are provisioned via Rippling for US employees. However, new hires have a grace period in which they are allowed to use their personal machines while awaiting a dedicated device from Rippling. International hires, as well, are not issued machines provisioned from Rippling, but are instead entrusted to purchase an off-the-shelf machine and install the Rippling agent, themselves.

These gaps in management of the devices reduce the efficacy of Rippling software, although not in a very meaningful way. However, allowing the usage of personal machines while awaiting a dedicated device presents some security risk, as personal machines are not guaranteed to be secure, and installation of Rippling may not retroactively address security issues present on personal devices.

Status	Issue	Control	Comments	Impact
MITIGATED	Endpoint compromise mitigation	Full disk encryption, regular backups, remote wipe, password and security config requirements[1]	Threats include ransomware, physical theft, seizure, and complete OS takeover. Rippling covers all of these controls	*MEDIUM*
UNMITIGATED	Security practices of employees	Workstation security training	Security training should occur at onboarding and on a periodic (annual) basis	*LOW*
UNMITIGATED	Usage of personal machines to access company resources	Machines should be company provisioned	Usage of personal machines downgrades the security of endpoints[2]	*LOW*

Status	Issue	Control	Comments	Impact
MITIGATED	Malware protection on endpoints	Standardize anti-virus software, disallow installation of unnecessary 3rd party software, OS security configurations, provide a method for sandboxing when opening external files	Malware protection is mostly about prevention. Employees should be trained to avoid running untrusted applications. Providing a sandbox for running necessary untrusted software is highly encouraged	*LOW*
MITIGATED	Network security	VPN requirement for all endpoints to access sensitive resources, minimalistic firewalls on all networks	<REDACTED> leverages a mostly trustless network and is not impacted by network threats[3]	*VERY LOW*

[1] Endpoints should require a strong account password, short screen lock times, disallowing installation of unsigned/untrusted software.
[2] Personal machines are only as secure as employees have kept them. Personal devices are not guaranteed to be handled at a high security level that company asset access requires.
[3] Network security is not necessary if services are authenticated, E2E encrypted, and require 2FA. There is some risk on employees using insecure local networks (e.g. public wifi at cafes or airports); a VPN is recommended when connecting to these insecure networks.

Secrets and Access Management

Secrets management is critical to organization security. Providing a secure storage for secrets is important for mitigating secrets compromise. Enforcing the principle of least privilege on secrets and service access mitigates insider threats and reduces impact of endpoint compromise.

Rippling's RPass software is currently used for endpoint password management. RPass is adequate for secrets distribution, but has a very poor user experience, making it less effective than other, well developed products. It is recommended to explore other password management software. The issues of RPass as a primary password manager are as follows:

- Only available as a Chrome browser extension (no cross-browser or desktop support)
- Personal passwords irrecoverable if vault password forgotten
- Poorly reviewed (slow and buggy, poor responsiveness and field detection)

Edit: <REDACTED> is now in the process of migrating password management largely from RPass to 1Pass. ShiftLeft is now being used for additional credential scanning. And, Doppler is being explored for secret distribution, and is highly recommended due to the fine-grained authorization capabilities it enables.

Status	Issue	Control	Comments	Impact
AT RISK	Permissions management	Enforce the principle of least privilege[1] on all services	Rippling is used to manage access to services, but fine-grained permissions are not well defined, and much user access is generally overly permissive	*MEDIUM*
IN PROGRESS	Poor local password management and secret storage	Provide a standard password manager on all endpoints, require secret sharing be done over encrypted and authenticated channels only	Rippling provides some password management via RPass, but is a suboptimal solution[2]. A dedicated solution is recommended[3]	*MEDIUM*
MITIGATED	2FA on all sensitive services	Enforce two-factor authentication on all services	This is the current policy for GitHub and Google Workspaces. 2FA is also recommended for all other services that support it	*MEDIUM*

Status	Issue	Control	Comments	Impact
MITIGATED	Repository visibility	Make repositories private	<REDACTED> repos are private by default	*MEDIUM*
AT RISK	Immediate access revocation	Immediate revocation of employee access, upon termination or compromise	Slack, email signing cert, GCP access, Github access	*LOW*
AT RISK	Non-segregation of production and non-production secrets, accounts, and permissions	Segregate prod and non-prod accounts and access	Development, non-prod secrets are currently checked in to git repos	*LOW*
PARTIALLY MITIGATED	Secrets (e.g. API tokens) not all stored in a secret manager	Secrets check-in detection via automated secret scanning on commit	Prod and non-prod assets should both be stored in a secret manager. Secret check-in scanners should not be bypassed[4]	*LOW*
MITIGATED	Immediate access revocation	Rippling provides adequate immediate revocation	The ability to perform immediate revocation is important for incident response and insider threats	*LOW*
MITIGATED	Authentication via SSO	Leverage SSO wherever possible	Rippling supports SSO with OpenID Connect support	*VERY LOW*
UNMITIGATED	Permissions creep / lingering access	Periodic permissions review[5]	This is a nice-to-have control for enforcing the principle of least privilege	*VERY LOW*

[1] "The principle [of least privilege] means giving a user account or process only those privileges which are essential to perform its intended function." - This principle is paramount in mitigating insider threats.

[2] RPass has many flaws (poor UX, only available as a browser extension, lacking important features such as 2FA) and is poorly rated. It is adequate for initial password distribution, but should not be offered as a primary password management tool.

[3] A poor user experience in a password manager will translate to less usage by employees. A mature password manager with good usability such as LastPass or 1Pass is recommended.
[4] Non-prod credentials have been checked in (development branch build secrets), however, it is still recommended to handle test and dev secrets in a secure manner.
[5] Individual employee permissions should be regularly evaluated (by each employee), and removed if they are no longer necessary

Application Development and Deployment

This report largely analyzes just Operational Security, but there is some overlap with Application Security and Development Operations. This section covers that overlap. It is highly recommended that <REDACTED> perform separate assessments of their AppSec and DevOps security on a regular basis.

<REDACTED> utilizes a CI/CD pipeline that operates as follows:

1. Developer commits to development, non-prod branches
2. Developer performs a pull request to merge to main branch
3. If tests pass and another developer has reviewed the changes, the merge succeeds and the main branch is updated
4. The project is tested and built
5. On successful build, a github action triggers a Google Cloud Build event
    a. Cloud build is authenticated via the Cloud Build GitHub app with workload identity federation by using OpenID Connect. Certain GitHub repos are whitelisted for deployments to Google Cloud
6. Google Cloud Build distributes the binary (docker image) to production hosts via App Engine

Securely configured CI/CD pipelines allow developers to be completely disconnected from operations that would require they have advanced permissions (committing directly to production branches, and deploying to production hosts). <REDACTED> fails to secure their CI/CD pipelines to an extent that prevents developers from circumventing security controls.

Beyond the pipeline, itself, there is a huge issue with GitHub and Google Cloud admin administrator permissions. Many employees have complete access to Google Cloud assets, rather than fine-grained, controlled permissions via custom IAM rules. Google Cloud admin permissions are an extreme level of access that should be very tightly controlled. GitHub admin access is also overly broad, allowing those admins the ability to defeat security controls for code repos.

Status	Issue	Control	Comments	Impact
AT RISK	Overly permissive deployment permissions & production access in Google Cloud	Restrict deployment permissions to automated systems only, with absolute minimal manual access to production assets	This is a significant issue for <REDACTED>. Giving many employees the ability to deploy arbitrary executables to production servers[1] and access prod assets is very risky	*CRITICAL*

Status	Issue	Control	Comments	Impact
AT RISK	Overly permissive GitHub permissions	Restrict GitHub admin permissions to a small set of users. Create alerts for any and all changes to the GitHub org or to individual repos	Anyone with GitHub owner permissions can override existing security controls and/or covertly modify code	*HIGH*
UNMITIGATED	Plaintext keys checked in to code repositories	Secret check-in prevention (git commit hook[2]), store ALL credentials in a secrets manager	Sonarcloud scans git repos for credentials, but non-prod development credentials have been checked in[3]	*MEDIUM*
MITIGATED	Unreviewed deployment branch commits	Enforce code reviews before merging changes into Main branches and disable direct commits to Main branches	Main branches require two peer reviews and passing tests on pull requests before merges are committed	*MEDIUM*
AT RISK	Repudiation of code commits	Reject unsigned commits	Commit blames could be impersonated	*LOW*
AT RISK	Unreviewed staging branch commits	Disable direct commits to main and staging branches and enforce pull requests to promote from test/development branches	Pre-production testing happens on the staging branches. Test scripts run on GitHub runners as part of this build/deployment cycle, allowing unreviewed test code to run with some privileges	*LOW*
MITIGATED	Dependency vulnerabilities	Vulnerability scanning on check-in (dependency scanning)	Dependabot and Snyk do CVE detection of dependencies	*VERY LOW*

[1] After committing to a main branch, developers can locally run a deployment script (that can be freely edited) that has permissions to push built code to Google Cloud servers. Github actions

can run the deployment scripts, as well, and have the same level of permission - GCP uses a "Service account impersonation" setup with GitHub to grant these permissions.

[2] https://github.com/awslabs/git-secrets is an example of a commit scanner for alerting on credential check-in.

[3] Development secrets are all exposed in application configs (unlike prod secrets).

Service Configurations

Services in use by <REDACTED> should be configured to enforce the principle of least privilege. Enforcing this principle leads to a reduced attack service from insider threats, asset compromise, and information leakage.

This report has identified some select services with overly broad access, but is not an exhaustive coverage of all service configurations at <REDACTED>. An in-depth analysis of the access configurations of all services in use by <REDACTED> should be performed in order to ensure that the principle of least privilege is applied.

Status	Issue	Control	Comments	Impact
AT RISK	Overly broad ownership permission of Google Cloud assets	Restrict admin permissions to a minimal set of users, configure IAM properly on each service	Includes Secrets Manager, IAM, Cloud SQL, App Engine, Compute Engine, etc.	*CRITICAL*
~~AT RISK~~ RESOLVED	Nansen.ai leaked credentials	Remove credentials and rotate password	Credentials are publicly visible on this wiki page	*MEDIUM*
~~AT RISK~~ RESOLVED	Notion wiki public access	Configure Notion wiki assets to be private only	Non-public information was published and publicly viewable	*LOW*
AT RISK	Notion wiki overly broad internal access	Lock pages to minimum access on a need-to-know basis	There is significant cross-team information leakage via the internal wiki	*LOW*
AT RISK	Admin access to <REDACTED> website	Reduce admin access to website assets to a small group of necessary admins	Admin access should always be tightly controlled to prevent potential abuse	*LOW*
PARTIALLY MITIGATED	Overly broad access to shared documents in Google Docs	Make google docs private by default, configure access control groups for fine-grained access sharing to groups	Shared docs through are reportedly not tightly access controlled on a need-to-know basis	*LOW*

Status	Issue	Control	Comments	Impact
AT RISK	Database Query Tool access[1]	Reduce access to Query Tool to only those with a business need	Everyone at <REDACTED> has access to this by default, granting read access to production data	*VERY LOW*

[1] This also includes the data visualization tool, and any other database access tools (such as Tableau), which have overly broad access permissions.

Insider Threats

An insider threat is a threat that presents through the vector of an employee (insider), allowing a malicious party to abuse the trust and permissions afforded to that insider. This includes the standard “rogue employee” risk model, which is often overlooked, but also includes the risk of an attacker compromising an employee.

Employees can be compromised in many different ways: Endpoint compromise through malware or poor physical security, physical duress (threats to safety), financial incentives (large black market bounties), social engineering, and through accidental information leakage.

It is highly recommended to reduce unnecessary employee permissions and put in place detective measures to detect potential compromise to mitigate the impact of potential insider threats.

Status	Issue	Mitigation	Comments	Impact
UNMITIGATED	Multi-party approval schemes for critical business functions	Implement approval systems for deployments, published content, and business functions	Multi-party integrity (via approvals or reviews) is a very strong tool for mitigating risk of abuse of business functions[1]	*MEDIUM*
AT RISK	Detection and alerting of production changes	Alteration of configurations and applications should trigger alerts to all admins for transparency[2]	Git keeps a record of code changes, but manual executable and config changes on the hosts are not monitored	*MEDIUM*
PARTIALLY MITIGATED	Employee access revocation process / lag time / runbook	Maintain an inventory of all employee permissions, allowing immediate revocation of access[3]	Endpoint compromise, account takeover, and termination are all reasons to revoke access immediately	*MEDIUM*
AT RISK	Lack of non-repudiation of business function blame	Watermark assets where possible, keep record of access events of sensitive information	<REDACTED> does not appear to have secret company information that would be at risk	*VERY LOW*

Status	Issue	Mitigation	Comments	Impact
UNMITIGATED	Alerting on out-of-cycle code commits	Out-of-cycle commits and changes should trigger alerts. Releases should be merged with main branches on a regular cadence	Performing commits and deployments at a regular cadence allows you to to monitor for out-of-cycle commits, which can be indicators of compromise	*VERY LOW*

[1] Business functions include publications, code commits, social media posts, interaction with HR/financial software, communications with business partners, etc.
[2] Transparency of production changes is a strong detective control for insider threats and host or endpoint compromise.
[3] Rippling performs these actions on a subset of services, but some other services are manually access controlled, and should be supplemented with a runbook for easily enumerating and immediately revoking employee permissions.

Social Engineering, Phishing Prevention, Information Leakage

Social engineering is the simplest path to compromise for an attacker, and attack vectors such as phishing are difficult to mitigate at scale. Research shows that those in technical roles are no less likely than those in non-technical roles to fall for phishing attacks. Everyone is vulnerable.

Mitigating the impact of compromise is important (as mentioned in the Insider Threats section), but phishing and social engineering can be mitigated at the employee level with proper training. Technical controls can provide partial mitigation, but phishing can only be significantly mitigated through adequate training and phishing challenges.

Phishing challenges are a challenge of your company's phishing readiness. Fake phishing messages should be periodically sent to employees, with tracking on what percentage of people click links, enter sensitive information, or fail to report phishing attempts.

Some basic email phishing indicators are:

- Hyperlinks that do not match the text of those links
- 'From' email field is spoofed (solved by email signature verification)
- Emails with attachments (files should be sent via an authenticated doc sharing platform)
- A conveyed sense of urgency or time-sensitivity
- Any request for credentials or permissions

Status	Issue	Mitigation	Comments	Impact
AT RISK	Lack of phishing awareness / social engineering training	Deliver periodic anti-phishing training[1]	Phishing training is the most powerful protection against one of the most common compromise vectors	*MEDIUM*
~~AT RISK~~ RESOLVED	Non-public information in public wiki	Wiki should be made entirely private, or all pages should be evaluated against visibility settings	This page seems to contain potentially non-public data, for example	*LOW*

Status	Issue	Mitigation	Comments	Impact
~~AT RISK~~ RESOLVED	Employee locations publicly accessible	Make the Team Member Locations page private	This page is publicly accessible and poses a safety and security risk	*LOW*
UNMITIGATED	Unverified external interactions	Verify external parties and content before trusting[2]	Interactions include bug reports, file sharing outside of the company, and business critical information over email or unauthenticated channels	*LOW*
UNMITIGATED	Anti-phishing email server controls	Remote content load prevention, signature verification, flagging masked URLs	Gmail provides basic phishing security controls, but can be configured to add extra protections[3]	*LOW*
UNMITIGATED	Lack of information disclosure non-repudiation measures or detection measures	Watermarking of sensitive documents, images, and videos; logging document access events	This applies mainly to highly sensitive documents and trade secrets	*VERY LOW*

[1] Here is a good resource on implementing phishing training.
[2] Verification can be done by collecting email signing certificates, providing an authenticated secure file share mechanism, or by confirming details over a separate secure channel before trusting them.
[3] All advanced security settings for Gmail should be enabled.

Wallet Security

<REDACTED> blockchain assets seem to be well protected and were left out of scope of this review, but the following general guidance for cryptocurrency wallet management has been included for good measure:

1. Use a hard wallet and actually read the tx data you are signing with it. This is the absolute best defense for your assets, and this step is impossible to defeat if you do it correctly. If you are extra paranoid or transacting on a potentially insecure network, verify expected TX data on multiple devices and networks in case any one of those are compromised.
2. Use an air-gapped, clean machine for interacting with your wallet. This is highly recommended when interacting with your cold wallet to reduce attack surface via endpoint compromise. QR code exchange can be useful for maintaining the air gap when transacting.
3. Don't store your seed phrase in any digital form. This should only ever touch your hard wallet and nothing else after generation/splitting. Use a multi-sig wallet OR split your seed phrase into an N of M set of secrets using Shamir's Secret Sharing algorithm and distribute the secrets to trusted custodians and/or store in various locations. Establish a key ceremony process for provisioning new wallets and testing backup recovery.
4. Use a hardened browser that is solely for web3 and initiating transactions. Follow a professional guide for hardening your specific browser and turn off every feature you don't need. Make sure to do security updates immediately, but skip feature updates.
5. Use a hot wallet for regular transacting and a cold wallet for long-term holding.
6. Don't ever click links, always type in addresses to avoid homograph and masking attacks. Always verify the TLS certificate to make sure the website you are visiting is authentic, when transacting.
7. Malicious websites can perform clickjacking attacks with good timing or impersonate plugin popups. Metamask and password managers both can open pages in a separate tab; avoid using these because they can be spoofed and trick you into divulging credentials to attackers. Keep soft wallets locked at all times when not actively transacting.

A general use PC has so many attack vectors, you should pretty much always assume that it is compromised and rely on your hard wallet and seed phrase management to make attack impossible.

This report was produced by Indigo Blockchain Security for <REDACTED>, based solely on information provided by <REDACTED>. Indigo Blockchain Security provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Indigo Blockchain Security is available for questions or comments about any of the contents of this report. We can be reached at https://indigo.reviews/ or by any of the contact methods registered under indigo.eth.